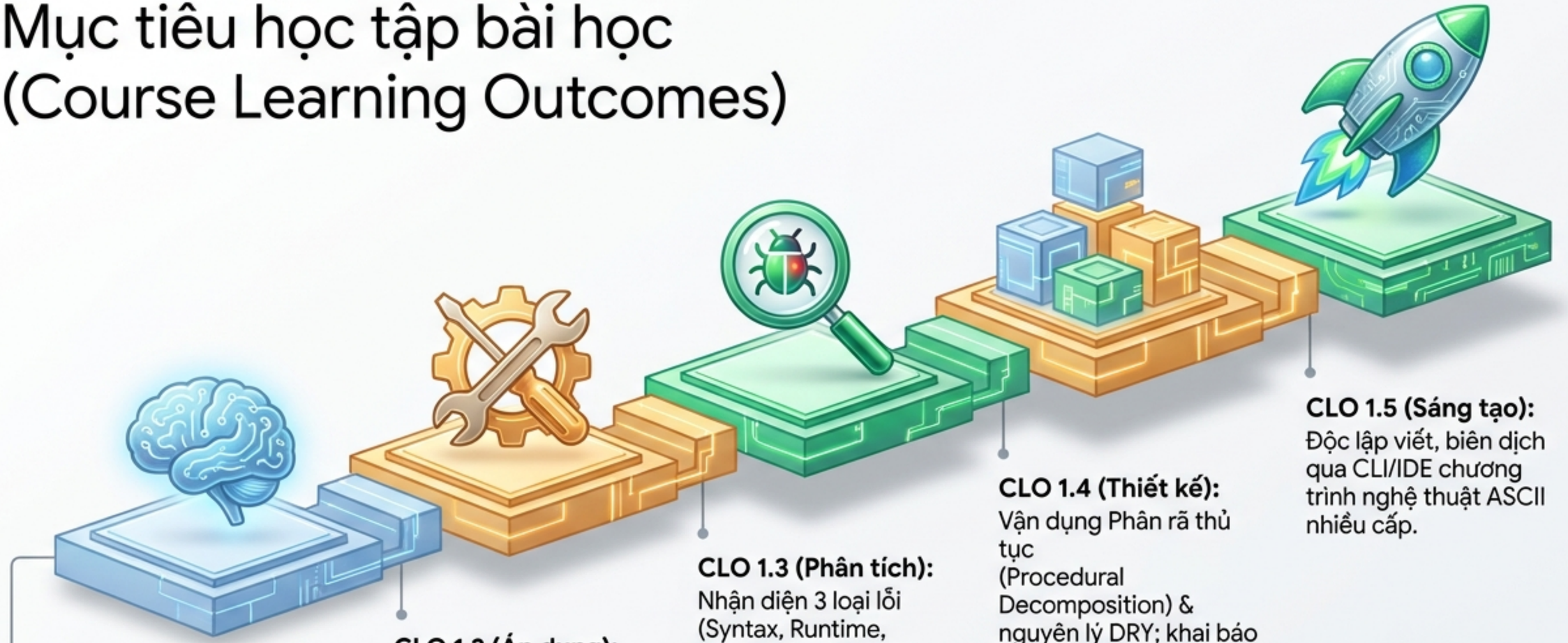
KỸ THUẬT LẬP TRÌNH / JAVA

Chương 1: Introduction to Java Programming

- Dành cho sinh viên Khóa 2026 – Lớp IT26M
- Khoa Công nghệ Thông tin
- Dựa trên giáo trình: Building Java Programs
- Buổi học tích hợp Lý thuyết & Thực hành tại lớp (Thời lượng: 3 Tiết)
- Năm học 2026

Gemini Notebook

Mục tiêu học tập bài học (Course Learning Outcomes)



CLO 1.1 (Hiểu):
Giải thích bản chất kiến trúc máy tính, hệ nhị phân, cơ chế biên dịch vs thông dịch, vai trò máy ảo JVM và mã Bytecode (WORA).

CLO 1.2 (Áp dụng):
Viết thành thạo khung chương trình Java cơ bản (class, main), phân biệt `print/println`, sử dụng ký tự thoát (`\n`, `\t`, `\t`, `"`, `\\`).

CLO 1.3 (Phân tích):
Nhận diện 3 loại lỗi (Syntax, Runtime, Logic); đọc thông báo lỗi từ trình biên dịch để debug.

CLO 1.4 (Thiết kế):
Vận dụng Phân rã thủ tục (Procedural Decomposition) & nguyên lý DRY; khai báo phương thức tĩnh (static); phân tích luồng điều khiển.

CLO 1.5 (Sáng tạo):
Độc lập viết, biên dịch qua CLI/IDE chương trình nghệ thuật ASCII nhiều cấp.

Gemini Notebook

Lộ trình buổi học 3 tiết (Tích hợp Lý thuyết & Thực hành)

TIẾT 1 (50 phút) - Nền tảng & Khởi đầu

- Khái niệm tính toán & Nhị phân
- Compiler vs. Interpreter
- Kiến trúc JVM & Bytecode
- Quy trình: Edit-Compile-Run
- Chương trình Hello.java

[Lab 1]: Viết & chạy Java trên CLI và IDE.

TIẾT 2 (50 phút) - Cú pháp & Gỡ lỗi

- Ký tự thoát (Escape Seq)
- Định danh, Từ khóa, CamelCase & Chú thích
- Phân loại 3 nhóm lỗi (Syntax, Runtime, Logic)
- Chiến lược đọc lỗi & gỡ lỗi

[Lab 2]: Thử thách Debug & Escape Strings.

TIẾT 3 (50 phút) - Phân rã & Case Study

- Tư duy phân rã Top-down & Phương thức tĩnh (static)
- Luồng điều khiển (Control Flow) & Nguyên lý DRY
- Case Study DrawFigures (Naïve -> Cấp 1 -> Triệt để)

[Lab 3]: Mini Project vẽ Tàu

Gemini Notebook

TIẾT 1: NỀN TẢNG HỆ THỐNG TÍNH TOÁN & KHỞI ĐẦU

Tại sao chúng ta cần học lập trình? (Why Programming?)

> Người ta thường nói rằng một người không thực sự hiểu điều gì đó cho đến khi họ dạy nó cho người khác. Trên thực tế, một người không thực sự hiểu điều gì đó cho đến khi họ dạy nó cho một chiếc máy tính, tức là diễn đạt nó dưới dạng một thuật toán.
>
> **— Donald Knuth**

Máy tính

Thiết bị phần cứng vạn năng (General-purpose device).

Chương trình (Program)

Tập hợp các chỉ thị ra lệnh cho phần cứng.

Lập trình

Rèn luyện tư duy giải quyết vấn đề có cấu trúc.

Gemini Notebook

Thành phần cốt lõi của máy tính

CPU: Bộ vi xử lý trung tâm. "Bộ não" thực thi các phép tính logic và số học.

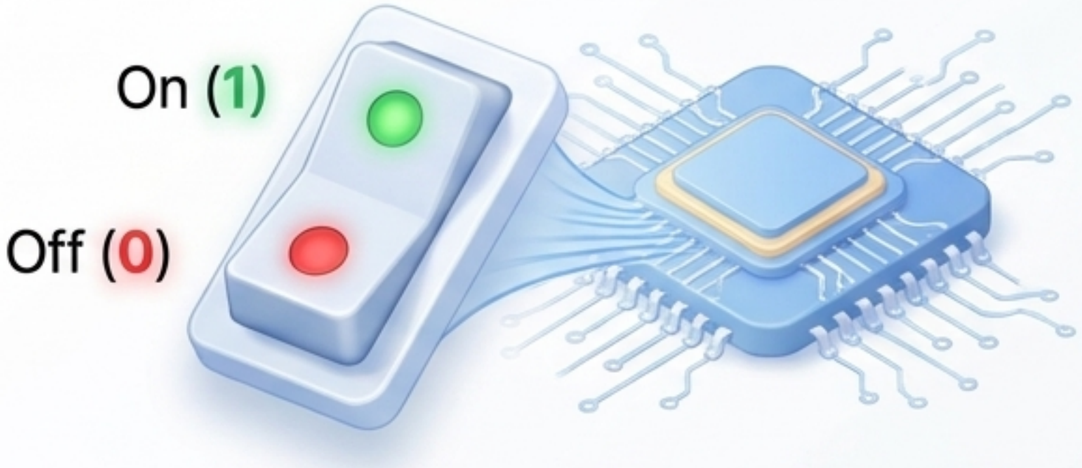
RAM: Bộ nhớ chính. Lưu trữ tạm thời, tốc độ cực nhanh, mất dữ liệu khi mất nguồn.

SSD/Ổ cứng: Bộ nhớ thứ cấp. Lưu trữ dữ liệu vĩnh viễn (persistent storage).

Thiết bị I/O: Bàn phím, chuột, màn hình, kết nối mạng. Giao tiếp với thế giới bên ngoài.

Gemini Notebook

Thế giới số & Biểu diễn dữ liệu bằng Hệ nhị phân (Binary)



Cơ chế biểu diễn số theo cơ số 2:

Trọng số	$2^3=8$	$2^2=4$	$2^1=2$	$2^0=1$
Bit	**1**	**0**	**1**	**1**

$(\mathbf{1011})_2 = 1\times8 + 0\times4 + 1\times2 + 1\times1$
$= 8 + 0 + 2 + 1 = \mathbf{11}$ (Hệ thập phân)

- Tại sao máy tính sử dụng số nhị phân? Phần cứng cấu tạo từ hàng tỷ bóng bán dẫn (transistors).
- Bật (On) / Điện áp cao = **1**.
- Tắt (Off) / Điện áp thấp = **0**.
- **Bit**: Đơn vị nhỏ nhất (0/1). **Byte**: 8 bits = 256 trạng thái (0-255).

Chốt lại: Mọi chương trình, hình ảnh, video cuối cùng đều được ánh xạ về dãy bit 0 và 1!

Gemini Notebook

Cơ chế thực thi: Trình biên dịch vs. Thông dịch

Trình biên dịch truyền thống (C/C++) (Compiler)

– **Dịch** toàn bộ mã nguồn thành Mã máy trực tiếp (Native Machine Code).
– **Ưu điểm:** Tốc độ thực thi cực nhanh, tối ưu phần cứng.
– **Nhược điểm chí tử:** Phụ thuộc nền tảng! (File Windows không chạy trên macOS/Linux).

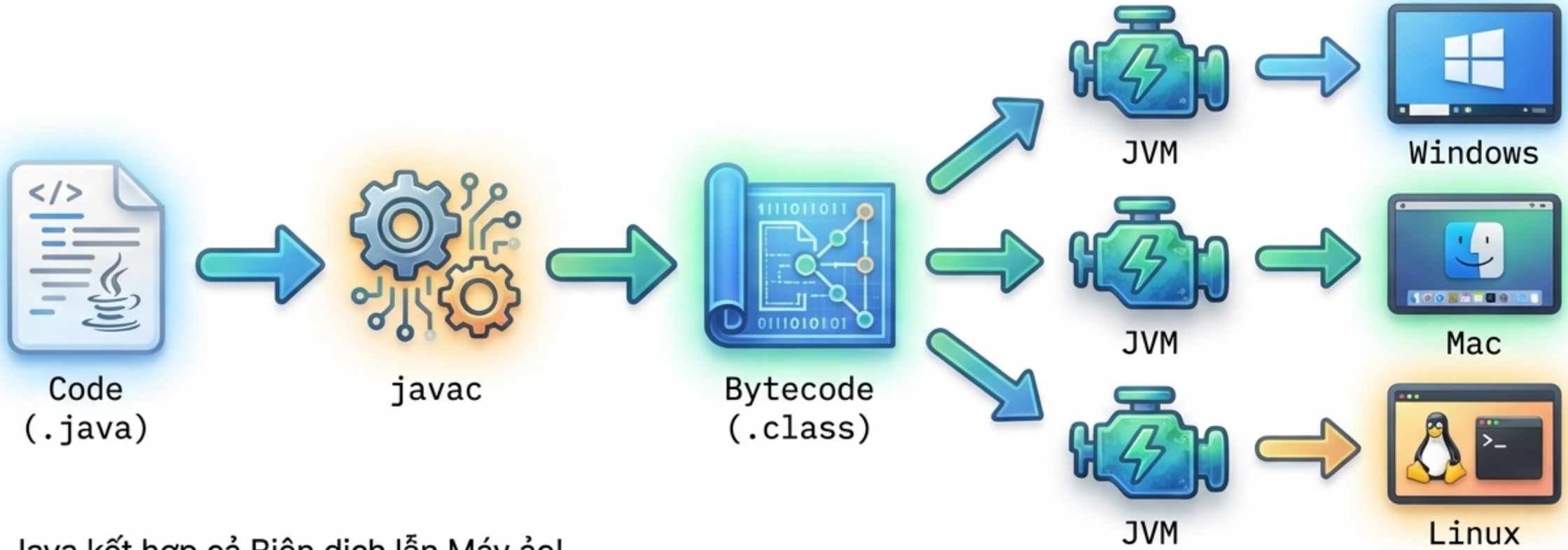
Trình thông dịch (Python, JavaScript) (Interpreter)

– Đọc và diễn giải trực tiếp từng dòng lệnh tại thời điểm chạy.
– **Ưu điểm:** Linh hoạt, dễ thử nghiệm trực tiếp.
– **Nhược điểm:** Tốc độ thực thi chậm hơn nhiều so với mã đã biên dịch sẵn.

Câu hỏi đặt ra cho Java năm 1995: Làm sao vừa đạt tốc độ thực thi cao, vừa chạy trên mọi nền tảng phần cứng mà không cần biên dịch lại?

Gemini Notebook

Triết lý: “Write Once, Run Anywhere” (WORA)

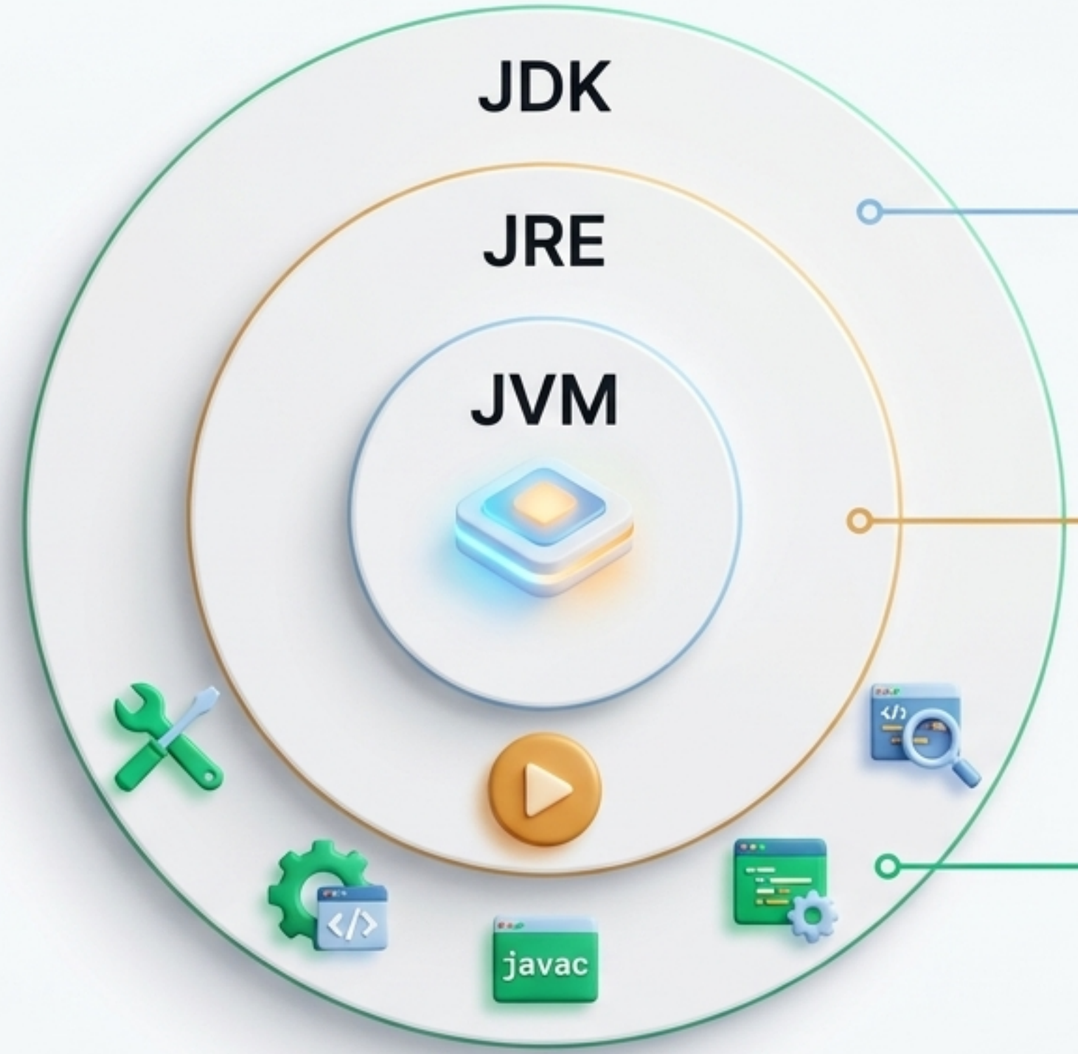


- Java kết hợp cả Biên dịch lẫn Máy ảo!
- Trình biên dịch (javac) không dịch ra mã máy thật. Nó dịch mã nguồn (.java) thành Java Bytecode (.class).
- Bytecode là tập lệnh chuẩn cho một cỗ máy lý thuyết gọi là Java Virtual Machine (JVM).
- Mỗi hệ điều hành có một bản JVM riêng để thông dịch/JIT-compile Bytecode ra mã máy phần cứng.

Gemini Notebook

Bộ công cụ Java:
Phân biệt JDK, JRE, JVM

`JDK = JRE + Tools`
`⊃ JRE = JVM + Libs`



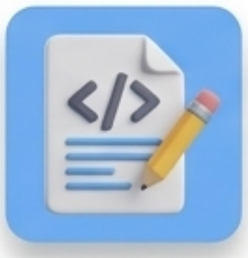
Java Virtual Machine:
Môi trường thực thi Bytecode.

Java Runtime Environment:
Gồm JVM + Thư viện chuẩn (Java Class Libraries) để người dùng chạy ứng dụng.

Java Development Kit:
Dành cho lập trình viên!
Chứa JRE + Trình biên dịch (`javac`) + Công cụ gỡ lỗi.

Gemini Notebook

Quy trình phát triển chương trình Java: Edit → Compile → Run



Bước 1: Soạn thảo (Edit)

Viết mã nguồn vào file văn bản thuần.
Bắt buộc đuôi file: `Hello.java`.

Bước 2: Biên dịch (Compile)

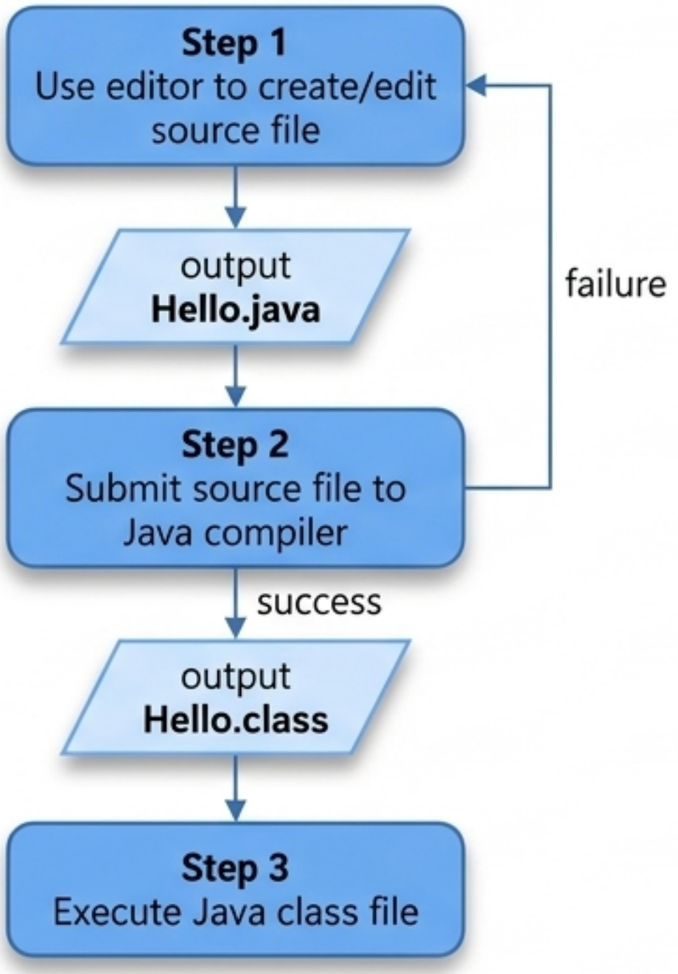
Lệnh: `javac Hello.java`

Kiểm tra lỗi cú pháp. Tạo ra file bytecode: `Hello.class`.

Bước 3: Thực thi (Run)

Lệnh: `java Hello`

Khởi động máy ảo JVM. JVM nạp bytecode, chuyển cho CPU thực thi. Kết quả xuất hiện trên màn hình!



Hình 1.1: *Quá trình tạo và thực thi chương trình Java*

Gemini Notebook

Chương trình Java hoàn chỉnh đầu tiên

Cấu trúc mẫu:
public class <TênLớp> { ... }

```java
// Chuong trinh Java dau tien cua lop IT26M
public class Hello
{
    public static void main(String[] args)
    {
        System.out.println("Hello, world!");
    }
}
```

Quy tắc vàng 1 (Ngoặc nhọn { }): Mọi mã nguồn phải nằm bên trong một lớp (class). Đóng gói phạm vi (scope). Mọi { mở phải có } đóng tương ứng.

Quy tắc vàng 2 (Tên file): Tên file bắt buộc trùng khớp 100% với tên lớp công khai. Lớp `public class Hello` ⇔ File `Hello.java`.

Quy tắc vàng 3: Java phân biệt chữ hoa và chữ thường (*case-sensitive*).

Gemini Notebook

Mổ xẻ phương thức **main** – Điểm bắt đầu của mọi ứng dụng

```
public static void main(String[] args) { … }
```

public | **Quyền truy cập công khai**. Bất kỳ đối tượng nào (bao gồm JVM) cũng có thể gọi phương thức này.

static | **Phương thức tĩnh cấp lớp.** Thực thi trực tiếp từ lớp mà không cần khởi tạo đối tượng (instance).

void | **Kiểu dữ liệu trả về.** Phương thức thực hiện công việc và không trả lại giá trị nào.

main | **Tên phương thức.** Tên quy ước đặc biệt mà JVM tìm kiếm đầu tiên khi chạy chương trình.

String[] args | **Mảng tham số dòng lệnh.** Cho phép truyền các chuỗi đối số từ bên ngoài khi khởi động chương trình.

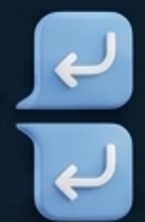
Gemini Notebook

Lệnh xuất dữ liệu: println vs. print

`System.out.println(...)`: In nội dung ra console và tự động xuống dòng mới (newline).

```
System.out.println("Dong thu nhat");
System.out.println("Dong thu hai");
```

```
Dong thu nhat
Dong thu hai
```



`System.out.print(...)`: In nội dung ra console và giữ nguyên con trỏ ở cuối dòng hiện tại.

```
System.out.print("Xin ");
System.out.print("chao ");
System.out.println("IT26M!");
```

```
Xin chao IT26M!
```

Lưu ý quan trọng! Mọi câu lệnh trong Java bắt buộc kết thúc bằng dấu chấm phẩy `;`. Lệnh `System.out.println()` không có tham số tạo ra một dòng trống.

Gemini Notebook

[CHECKPOINT 1] & [HANDS-ON LAB 1]

Câu 1: Lệnh hợp lệ?

✅ **Đáp án C:** System.out.println("Hello"); (Java phân biệt hoa thường, chuỗi trong nháy kép "")

Câu 2: Vai trò file .class?

✅ **Đáp án B:** Chứa mã Bytecode đã được biên dịch để JVM thực thi độc lập nền tảng.

HANDS-ON LAB 1: 15 Phút

1. Mở Terminal, gõ **javac -version** & **java -version**.
2. Tạo thư mục Lab1 & file **WelcomeIT26M.java**.
3. Biên dịch: **javac WelcomeIT26M.java**
4. Chạy: **java WelcomeIT26M**

```
public class WelcomeIT26M {
  public static void main(String[] args) {
    System.out.println("Chao mung lop IT26M den voi Java!");
    System.out.println("Viet mot lan, chay moi noi (WORA).");
  }
}
```

Gemini Notebook

TIẾT 2: Chuỗi ký tự, Ký tự thoát & Kỹ năng chẩn đoán, gỡ lỗi

Lập trình Java - Lớp IT26M

Thời lượng: 50 phút | 35’ Lý thuyết & Thảo luận + 15’ Hands-on Lab 2

Gemini Notebook

Lý thuyết:

Chuỗi ký tự (String Literal) là một chuỗi các ký tự nằm trong cặp dấu ngoặc kép (“...”).
Ví dụ: “Xin chao”, “IT26M”, “” (chuỗi rỗng).

Không được ngắt một chuỗi qua nhiều dòng mà không đóng ngoặc kép!

Ngữ cảnh: Nếu muốn in ra màn hình chính xác câu: Giao vien noi: “Java rat thu vi!”

Warning

main.java

```
1
2   System.out.println("Giao vien
      noi: "Java rat thu vi!");
3
```

Giải thích lỗi: Trình biên dịch sẽ báo lỗi ngay! Vì nó nghĩ chuỗi kết thúc ở dấu ngoặc kép thứ hai.

Gemini Notebook

Ký tự thoát chuẩn (Escape Sequences) - Quy tắc: Dấu gạch chéo ngược (\) kết hợp với ký tự đi liền sau.

	\t	**Tab**	Thêm khoảng thụt lề tab (căn chỉnh cột dữ liệu đẹp mắt).
	\n	**Newline**	Xuống dòng mới ngay tại vị trí xuất hiện.
	\"	**Double quote**	In dấu ngoặc kép " mà không làm đóng chuỗi.
	****	**Backslash**	In một dấu gạch chéo ngược ****.

Listing 3

main.java

```java
System.out.println("Cot 1\tCot 2\tCot 3");
System.out.println("Dong 1\nDong 2\nDong 3");
System.out.println("Dau \"nhay kep\" va gach \");
```

```
Cot 1    Cot 2    Cot 3
Dong 1
Dong 2
Dong 3
Dau "nhay kep" va gach \
```

Gemini Notebook

Định danh (Identifiers) – Tên do lập trình viên đặt cho lớp, phương thức, biến...

Tiêu chuẩn chung: Ký tự hợp lệ: Các chữ cái (**a-z, A-Z**), chữ số (**0-9**), dấu gạch dưới (**_**), dấu đô la (**$**).

Quy tắc 1: Không được bắt đầu bằng chữ số!

diem1

1diem

Quy tắc 2: Không chứa dấu cách hoặc ký tự đặc biệt (@, #, %, -, +).

Quy tắc 3: Không trùng với Từ khóa Java!

Gemini Notebook

Từ khóa chuẩn Java (Keywords – Dành riêng)

Java có 50+ từ khóa đặc biệt đã được định nghĩa sẵn cú pháp.

Grid Box

`public`	`class`	`static`	`void`
`int`	`double`	`boolean`	`char`
`if`	`else`	`for`	`while`
`return`	`new`	`true`	`false`

Tuyệt đối không dùng các từ trên làm tên Class hay tên hàm!

Gemini Notebook

Phong cách lập trình chuẩn đại học

Tên Lớp: UpperCamelCase: Viết hoa chữ cái đầu của mỗi từ.

Hello DrawFigures3 StudentManager

Tên Phương thức & Biến: **lowerCamelCase**: Chữ cái đầu viết thường, viết hoa các từ tiếp theo.

main drawDiamond calculateTotalScore

Thụt lề: Indentation: Nhất quán 4 khoảng trắng!

Gemini Notebook

3 Loại chú thích (Comments) trong Java

Chú thích 1 dòng

Cú pháp: `//`

```
// Tinh tong diem
cua sinh vien
```

Chú thích nhiều dòng

Cú pháp: `/* ... */`

```
/* Tac gia: IT26M
   Ngay tao: 2026 */
```

Chú thích tài liệu (Javadoc)

Cú pháp: `/** ... */`

Dùng để sinh tài liệu API tự động.

“Mã nguồn viết ra cho người đọc hiểu trước, máy tính hiểu sau.”

Gemini Notebook

Phân loại lỗi lập trình: 3 nhóm lỗi kinh điển

Mọi lập trình viên từ sinh viên năm nhất đến chuyên gia thâm niên đều phải đối mặt với lỗi!

Cú pháp - Syntax Errors

Thời điểm: Biên dịch (Compile-time). Trình biên dịch javac từ chối sinh file .class.

Đặc điểm: Vi phạm ngữ pháp (thiếu dấu ;, sai từ khóa, quên đóng }).

Thực thi - Runtime Errors

Thời điểm: Đang chạy (Run-time). Chương trình bị sập (crash) và tung ra ngoại lệ (Exception).

Đặc điểm: CPU thực hiện phép toán bất khả thi (chia cho 0, truy cập mảng rỗng).

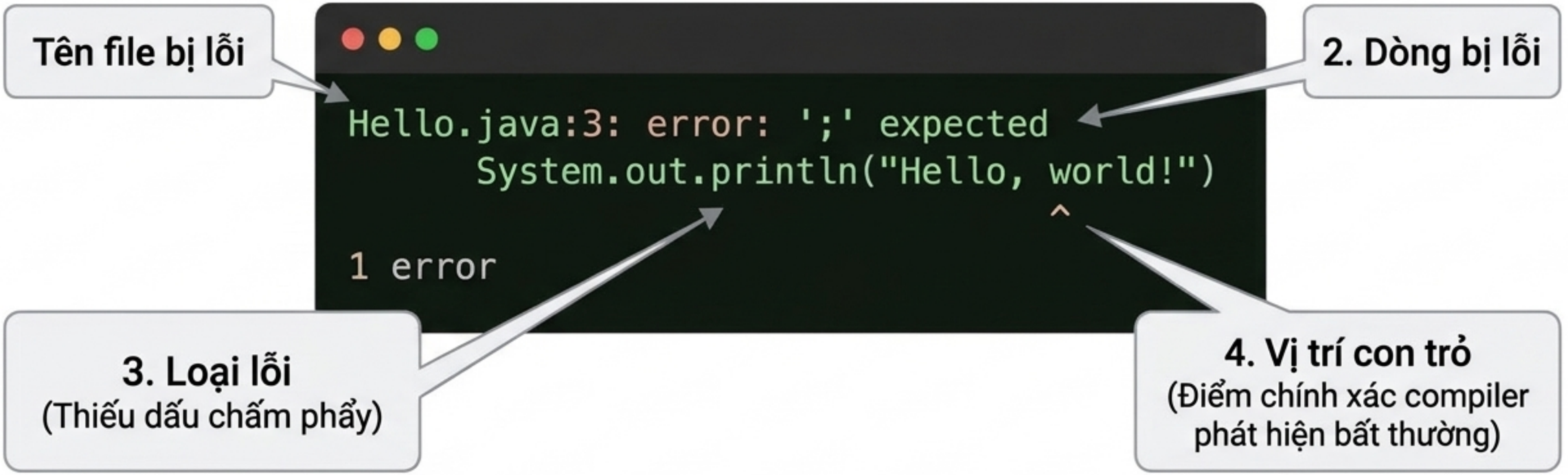
Ngữ nghĩa - Logic Errors

Thời điểm: Chạy không sập, nhưng kết quả sai.

Đặc điểm: Nguy hiểm nhất vì máy tính không thể tự phát hiện! (VD: Cần tính tiền thuế mà viết nhầm dấu + thành dấu -).

Gemini Notebook

Kỹ năng đọc thông báo lỗi từ Trình biên dịch



Kinh nghiệm vàng khi Debug

- **Quy tắc 1:** Luôn nhìn vào lỗi đầu tiên! Một lỗi ở dòng 3 có thể tạo 10 lỗi giả phía sau.
- **Quy tắc 2:** Nếu dòng báo lỗi trông hoàn toàn đúng, hãy kiểm tra dòng ngay phía trước nó.

Gemini Notebook

[CHECKPOINT 2] Thử tài thám tử chẩn đoán lỗi

```
1    public  MyProgram {
2        public static void Main(String[] args) {
3            System.out.println("Lop IT26M);
4        }
5    }
```

Chẩn đoán và phân tích chi tiết:

1. **Lỗi dòng 1:** Thiếu từ khóa class → Sửa: **public class** MyProgram.
2. **Lỗi dòng 2:** Tên hàm main bị viết hoa chữ M (Java phân biệt hoa thường) → Sửa: **main**.
3. **Lỗi dòng 3:** Chuỗi ký tự thiếu ngoặc kép đóng " và dấu chấm phẩy ; → Sửa: System.out.println(**“Lop IT26M”**);.

Gemini Notebook

[HANDS-ON LAB 2] Thực hành Debug & Chuỗi ký tự thoát

Bài toán: Viết chương trình EscapeChallenge.java (15 phút).

Gợi ý kỹ thuật:

- Đường dẫn: Dùng \\.
- Khẩu hiệu: Dùng ".
- Thụt lề: Dùng \t.

Thử thách: Cố tình gõ thiếu dấu \ để xem trình biên dịch báo lỗi gì!

```
Dac ta he thong:
    1. Thu muc:  C:\Java\IT26M
    2. Khau hieu: "Hoc la phai hanh!"
    3. Tam giac:  /\ /\
```

Gemini Notebook

TIẾT 3: Phân rã thủ tục, Phương thức tĩnh & Tiến hóa Case Study

Tư duy Kiến trúc (Procedural Decomposition)

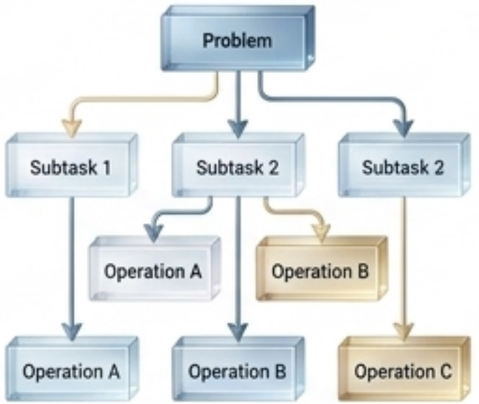
Thời lượng: 50 phút | 30’ Lý thuyết thiết kế + 20’ Mini Project Lab 3

Gemini Notebook

Cạm bẫy của chương trình nguyên khối - Monolithic

- Tất cả hàng trăm dòng lệnh trong hàm main.
- Mã nguồn cực kỳ khó đọc (spaghetti code).
- Trùng lặp mã nguồn (Code Duplication): Gõ đi gõ lại, khó sửa lỗi.

Phân rã Top-down - Divide and Conquer



- Chia bài toán lớn thành các bài toán con (subtasks).
- Tiếp tục chia nhỏ đến các thao tác đơn giản nhất.
- Ví dụ: Sơ đồ “Make Cake” (Chia thành Làm bột, Nướng, Làm kem).
- Mỗi hộp trong sơ đồ = Một phương thức tĩnh (static method) trong Java.

Nguyên lý cốt lõi: DRY – Don’t Repeat Yourself!

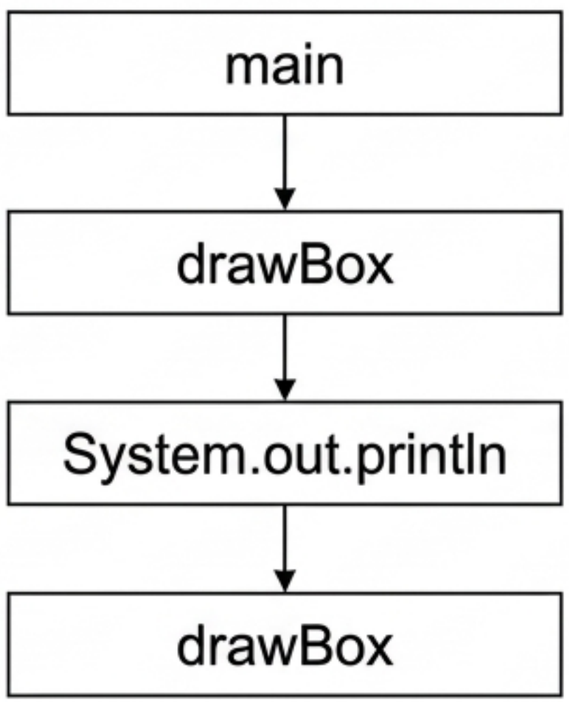
Gemini Notebook

Cú pháp **Khai báo** - Định nghĩa hành vi:

```
1  public static void drawBox() {
2      // Các câu lệnh in hình hộp
3  }
```

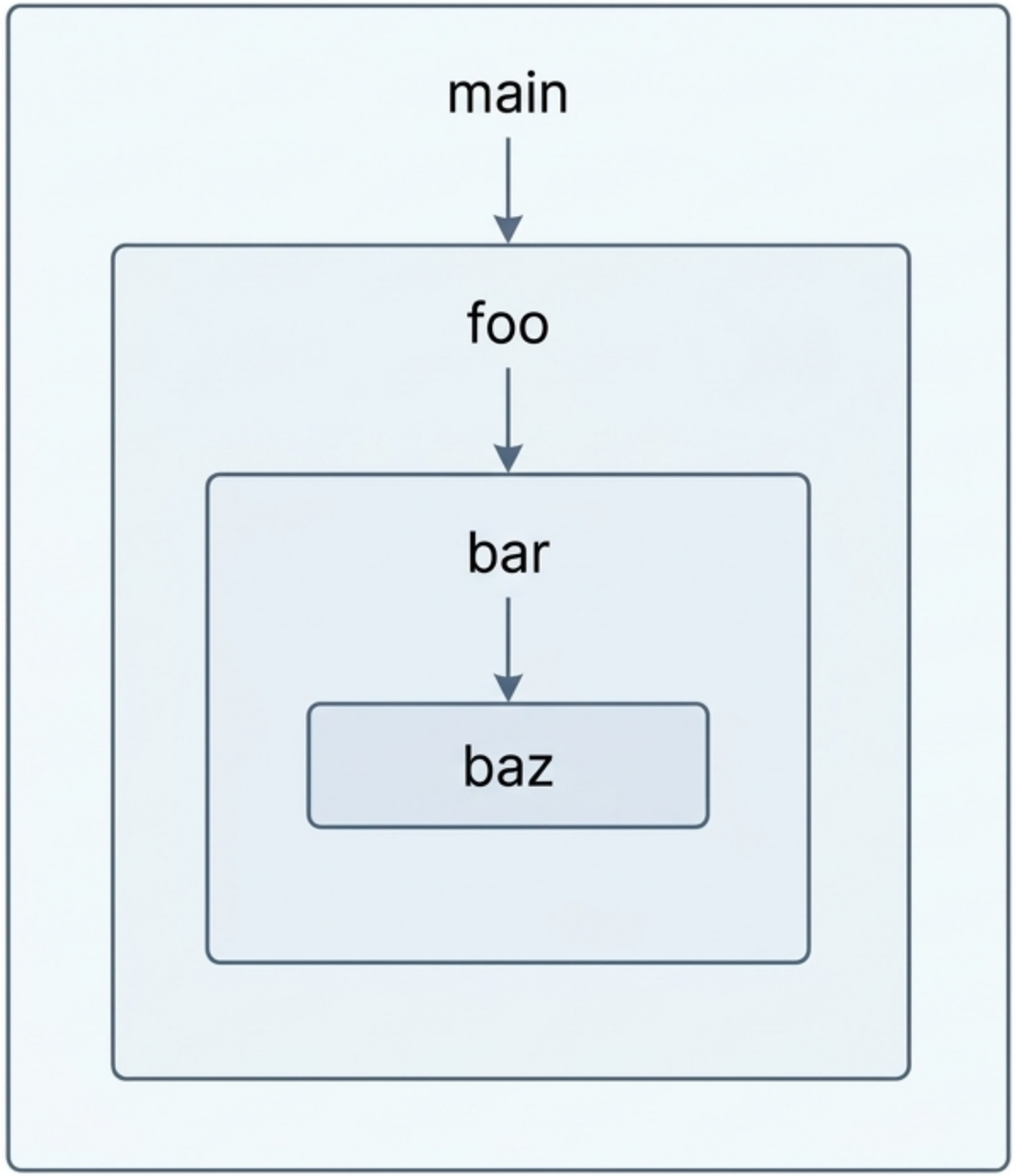
Cú pháp **Lời gọi** - Ra lệnh thực thi:

```
1  public static void main(String[] args) {
2      drawBox(); // Gọi lần 1
3      System.out.println();
4      drawBox(); // Gọi lần 2 (Tái sử dụng!)
5  }
```



Quy tắc chuyển giao:
Tạm dừng tại main → Nhảy tới **drawBox** thực thi → Quay về (**return**) đúng vị trí cũ để tiếp tục.

Gemini Notebook



Ngăn xếp gọi hàm (Call Stack)

- **Một phương thức có thể gọi phương thức khác.**
- Máy tính ghi nhớ thứ tự quay về bằng Call Stack **(LIFO: Vào sau ra trước).**
- Ví dụ: baz() chạy xong → quay về bar() → bar() chạy xong → quay về main.

Lợi ích kép của Phân rã thủ tục

1. **Structure** (Cấu trúc): Biến chương trình phức tạp thành module dễ hiểu.
2. **Eliminate Redundancy** (Xóa bỏ trùng lặp): Viết 1 lần, gọi thực thi vô số lần!

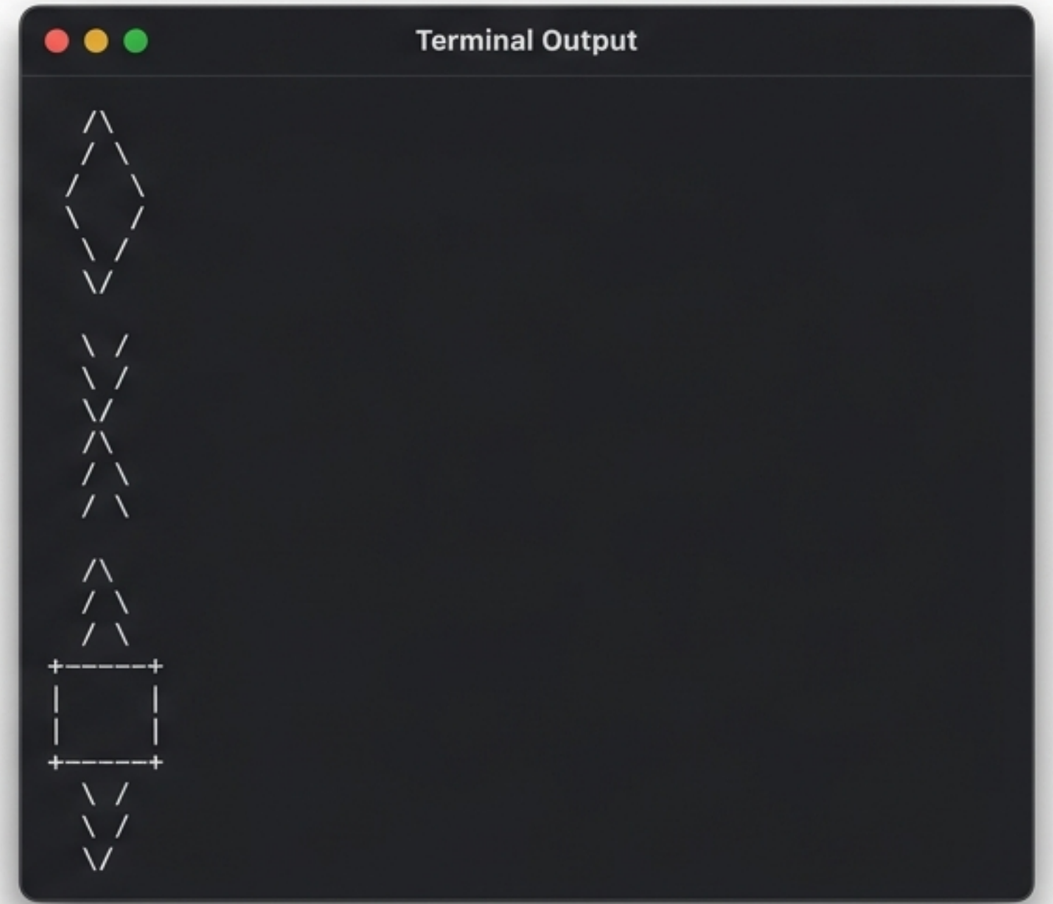
Gemini Notebook

Case Study: Bài toán vẽ hình ASCII nghệ thuật

The Problem

Viết chương trình in ra 3 hình đồ họa ký tự ASCII: Kim cương (Diamond), Chữ X (X), Tên lửa vũ trụ (Rocket). Các hình cách nhau một dòng trống.

The Output



Gemini Notebook

Phiên bản 1: Cạm bẫy của hướng tiếp cận ngây thơ (Naïve Approach)

DrawFigures1.java

```java
public class DrawFigures1 {
    public static void main(String[] args) {
        // Ve kim cuong (Diamond)
        System.out.println(" /\ ");
        System.out.println(" / \ ");
        System.out.println(" /   \ ");
        System.out.println(" \   / ");
        System.out.println(" \   / ");
        System.out.println(" \\/ ");
        System.out.println();
        // Ve chu X
        System.out.println(" \   / ");
        System.out.println(" \   / ");
        System.out.println(" \\/ ");
        System.out.println(" /\ ");
        // ... tiep tuc hang chuc dong lenh!
    }
}
```

Số dòng lệnh

Hơn 30 dòng lệnh `println` dồn hết vào một hàm `main` duy nhất.

Trùng lặp trầm trọng

Cụm nón trên (cone) lặp lại 3 lần.
Cụm nón dưới (v) lặp lại 2 lần.
Cụm hình hộp lặp lại 2 lần.

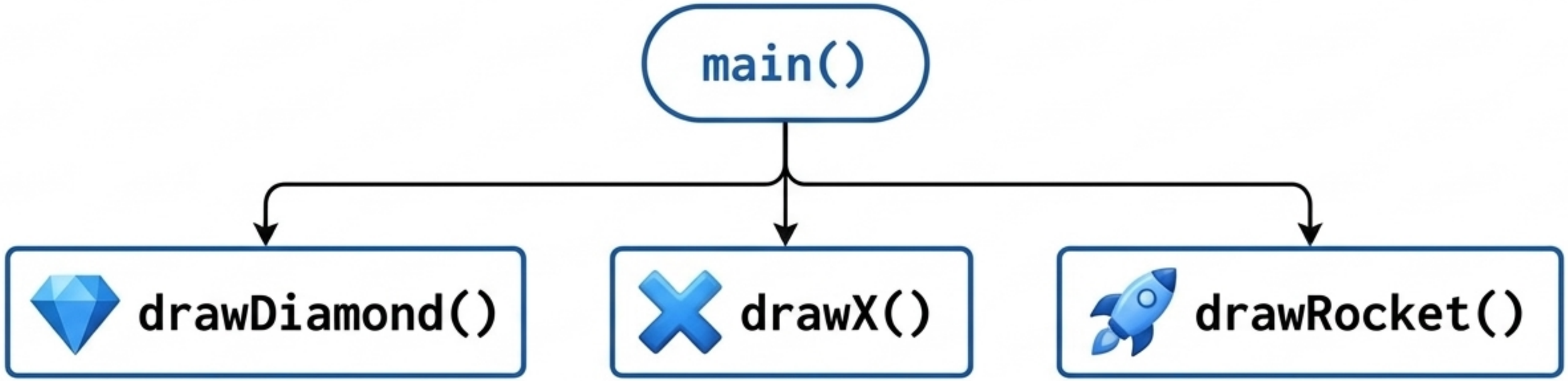
Hậu quả bảo trì

Cực kỳ rủi ro. Nếu muốn mở rộng hình, lập trình viên phải sửa thủ công ở hàng chục vị trí khác nhau.

→ Cần áp dụng Phân rã thủ tục cấp 1!

Gemini Notebook

Phiên bản 2: Phân rã Cấp 1 theo từng Khối hình



Cải tiến

Hàm main hiện tại cực kỳ ngắn gọn và sáng sủa:
drawDiamond();
drawX();
drawRocket();

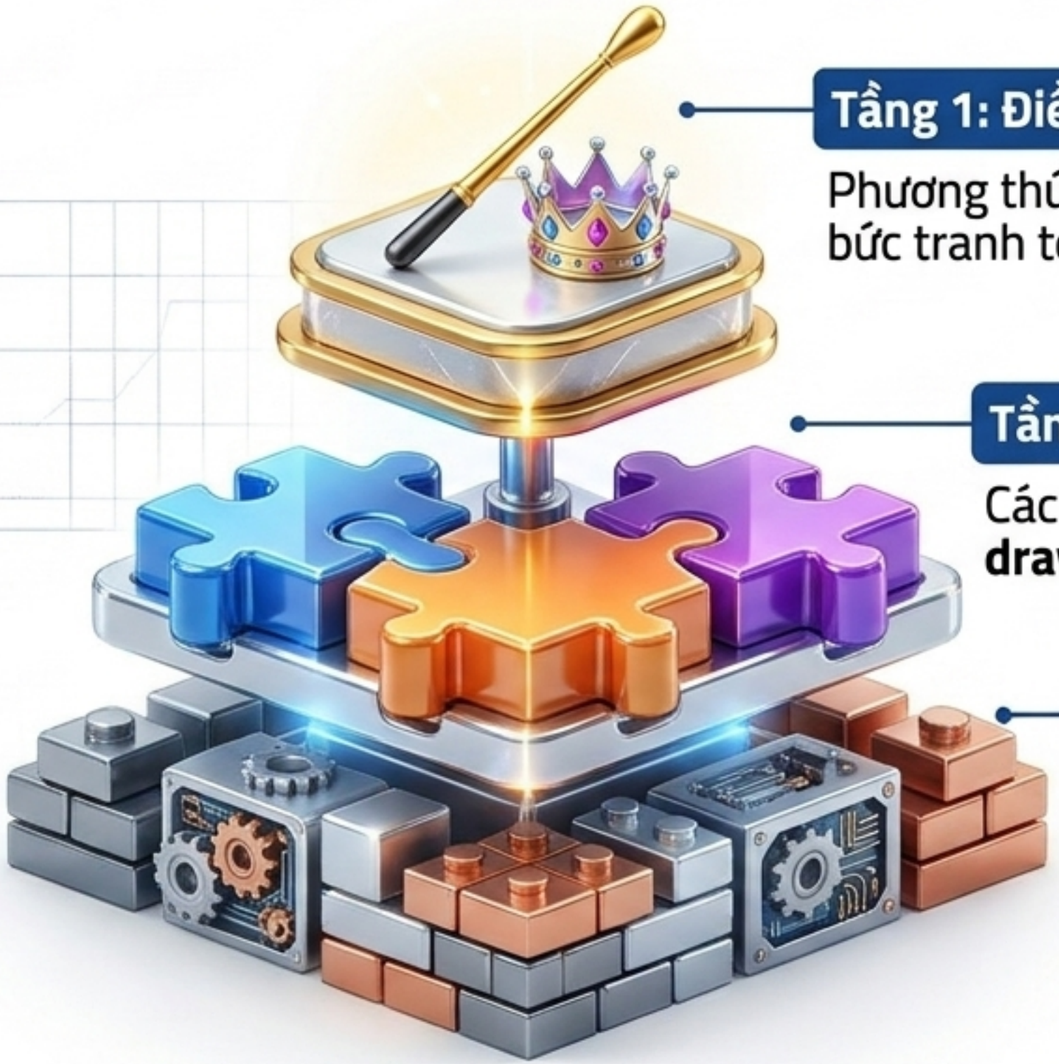
Hạn chế

Vẫn còn trùng lặp nội bộ. Bên trong **drawDiamond**, **drawX**, và **drawRocket**, các lệnh vẽ nón xuôi **(/\)** và nón ngược **(\/)** bị lặp lại nhiều lần.

Giải pháp triệt để: Phân rã tiếp xuống cấp thứ 2 thành các hình khối cơ sở (primitives).

Gemini Notebook

Phiên bản 3: Cấu trúc 3 tầng chuẩn kiến trúc phần mềm



Tầng 1: Điều phối

Phương thức **main()** nắm giữ bức tranh toàn cảnh.

Tầng 2: Khối hình hoàn chỉnh

Các phương thức **drawDiamond()**, **drawX()**, **drawRocket()**.

Tầng 3: Các khối nguyên thủy (Primitives)

Các phương thức **drawCone()** (Nón trên), (Nón trên), **drawV()** (Nón dưới), **drawBox()** (Thân hộp).

Triệt tiêu hoàn toàn trùng lặp thông qua khả năng **tái sử dụng.**

Gemini Notebook

Mã nguồn DrawFigures3: Tầng Điều phối và Khối hình

Tầng 1: Điều phối (main)

```java
public class DrawFigures3 {
    public static void main(String[] args) {
        drawDiamond();
        System.out.println();
        drawX();
        System.out.println();
        drawRocket();
    }
}
```

Tầng 2: Khối hình

```java
public static void drawDiamond() {
    drawCone();
    drawV();
}

public static void drawX() {
    drawV();
    drawCone();
}

public static void drawRocket() {
    drawCone();
    drawBox();
    drawBox();
    drawCone();
}
```

Gemini Notebook

Mã nguồn DrawFigures3: Tầng Khối nguyên thủy (Primitives)

Tái sử dụng 100% cho mọi hình

Tầng 3: Khối cơ sở (Primitives Reusable)

```
public static void drawCone() {
    System.out.println(" /\");
    System.out.println(" / \\");
    System.out.println(" / \\");
}

public static void drawV() {
    System.out.println(" \ /");
    System.out.println(" \ /");
    System.out.println(" \/");
}

public static void drawBox() {
    System.out.println("+-----+");
    System.out.println("|     |");
    System.out.println("|     |");
    System.out.println("+-----+");
}
```

Triệt tiêu hoàn toàn trùng lặp code in hình

Gemini Notebook

So sánh & Đánh giá 3 thế hệ thiết kế

Tiêu chí đánh giá	DrawFigures1 (Naïve)	DrawFigures2 (Cấp 1)	DrawFigures3 (Cấp 2)
Cấu trúc mã nguồn	❌ Rối rắm, nguyên khối 1 hàm duy nhất.	⚠️ Tách biệt từng hình lớn.	✅ **Phân tầng rõ rệt:** Điều phối - Lắp ráp - Nguyên tử.
Mức độ trùng lặp	❌ Rất cao (Heavy redundancy).	⚠️ Vẫn lặp code vẽ nón và vẽ hộp.	✅ **Triệt tiêu 100% trùng lặp code.**
Khả năng tái sử dụng	❌ 0% (chỉ có copy-paste).	⚠️ Chỉ tái sử dụng được cả khối hình lớn.	✅ **Tái sử dụng tối đa đa mọi khối nguyên thủy.**
Khả năng bảo trì	❌ Sửa 1 chi tiết phải đổi hàng chục chỗ.	⚠️ Vẫn phải sửa nhiều nơi.	✅ **Chỉ cần sửa 1 chỗ duy nhất!**

Gemini Notebook

[CHECKPOINT 3] Dự đoán luồng thực thi hàm lồng nhau

TraceTest.java

```java
public class TraceTest {
    public static void main(String[] args) {
        first();
        second();
    }

    public static void first() {
        System.out.println("1");
    }

    public static void second() {
        System.out.println("2");
        first();
        System.out.println("3");
    }
}
```

Thứ tự in trên màn hình là gì?

A. 1, 2, 3

B. 1, 2, 1, 3

C. 1, 2, 3, 1

D. 2, 1, 3

Giải thích chi tiết:

1. **main** gọi **first()** → in 1.
2. **main** gọi **second()**:
   - ↳ In 2.
   - ↳ Gọi **first()** → in 1.
   - ↳ In 3.

Gemini Notebook

[HANDS-ON LAB 3] Mini-Project Thiết kế chương trình

Lựa chọn A: SpaceRocket.java

Vẽ một trạm không gian gồm 2 tên lửa đứng cạnh nhau, mỗi tên lửa gồm mũi nhọn, thân chứa biểu tượng IT26M, khoang động cơ, và ngọn lửa đáy.

Yêu cầu: Triệt tiêu 100% trùng lặp.

Gợi ý cấu trúc:

```
drawNoseCone(),
drawBodySegment(),
drawThruster(),
drawFlame()
```

Lựa chọn B: FightSong.java

In lời bài hát có các điệp khúc lặp lại bằng cách phân rã thành ít nhất 2 phương thức tĩnh phụ ngoài main để loại bỏ trùng lặp văn bản.

Tiêu chí chấm điểm Lab:

✅ 0 errors, 0 warnings (Biên dịch hoàn hảo) | Tối thiểu 2 cấp phân rã | Tuân thủ chuẩn đặt tên CamelCase

Gemini Notebook

Tổng kết toàn bộ kiến thức Chương 1

Checklist Chuẩn đầu ra (CLOs) - Trạng thái: Đã làm chủ

CLO 1.1
Hiểu phần cứng, nhị phân, vai trò của Bytecode và máy ảo JVM (WORA).

CLO 1.2
Viết đúng cấu trúc class, hàm main, dùng thành thạo ký tự thoát (\n, \t, ", \).

CLO 1.3
Nhận diện 3 loại lỗi: Cú pháp (Syntax), Thực thi (Runtime), Ngữ nghĩa (Logic).

CLO 1.4
Nắm vững tư duy phân rã Top-down và ngăn xếp gọi hàm.

CLO 1.5
Áp dụng nguyên lý DRY, biên dịch và chạy độc lập trên CLI và IDE.

Gemini Notebook

Bảng tra cứu nhanh cú pháp cốt lõi (Java Cheatsheet)

Cấu trúc & Xuất màn hình

```
public class TenLop {
  public static void main(String[] args) {
    System.out.println("Nội dung");
  }
}
```

Khai báo & Gọi hàm

Khai báo:

```
public static void tenHam() { ... }
```

Lời gọi:

```
tenHam();
```

Escape Characters

\n	: Xuống dòng
\t	: Dấu Tab
"	: Dấu ngoặc kép
\	: Gạch chéo ngược

Quy tắc vàng

- Hàm không được lồng trong hàm khác.
- Đặt tên theo chuẩn lowerCamelCase.
- Triệt tiêu trùng lặp code (Nguyên lý DRY).

Gemini Notebook

Nhiệm vụ tự học & Bài tập về nhà

Bài tập bắt buộc
(Nộp trên LMS)

- Đọc lại Chương 1 (p.1-102)
- Self-Check: 6, 8, 10, 13, 18, 19, 24, 26, 27
- Lập trình: Ex 7 (Mantra), Ex 11 (TwoRockets)

Bài tập thử thách
(Cộng điểm)

- Project 1: In chữ MISSISSIPPI dạng khối lớn.
- Project 3: Bài hát dân gian tích lũy điệp khúc có cấu trúc phân tầng.

Chuẩn bị Buổi kế tiếp

- Đọc trước Chương 2: Primitive Data and Definite Loops
- Tìm hiểu: Biến, Kiểu số, Biểu thức, Vòng lặp for.

CẢM ƠN CÁC BẠN SINH VIÊN LỚP IT26M! HỎI ĐÁP & THẢO LUẬN (Q&A)

Chúc các bạn có những giờ học lập trình Java say mê và hiệu quả!

Gemini Notebook